

Improving our safety with a physical quantities and units library

Document #: P2981R1
Date: 2023-11-08
Project: Programming Language C++
Audience: SG23 Safety and Security
Reply-to: Mateusz Pusz ([Epm Systems](mailto:mateusz.pusz@gmail.com))
<mateusz.pusz@gmail.com>
Dominik Berner
<dominik.berner@gmail.com>
Johel Ernesto Guerrero Peña
<johelegp@gmail.com>

Contents

- 1 Revision history
 - 1.1 Changes since [P2981R0]
- 2 Introduction
- 3 Terms and definitions
- 4 The future is here
- 5 Affected industries
- 6 Mismeasure for measure
- 7 Common smells when there is no library for physical quantities
 - 7.1 The proliferation of double
 - 7.2 The proliferation of magic numbers
 - 7.3 The proliferation of conversion macros
 - 7.4 Lack of consistency
 - 7.5 Lack of a conceptual framework
- 8 Safety features
 - 8.1 Unit conversions
 - 8.2 Preventing truncation of data
 - 8.3 The affine space
 - 8.4 `explicit` is not explicit enough
 - 8.5 Obtaining the numerical value of a quantity
 - 8.6 Preventing dangling references
 - 8.7 Quantity kinds
 - 8.8 Various quantities of the same kind
 - 8.8.1 Converting between quantities of the same kind
 - 8.8.2 Comparing, adding, and subtracting quantities of the same kind

- 8.8.3 Modeling a quantity kind
- 8.8.4 Restricting units to specific quantity kinds
- 8.9 Non-negative quantities
- 8.10 Vector and tensor quantities
- 9 Safety pitfalls
 - 9.1 Integer division
 - 9.2 Lack of safe numeric types
 - 9.3 Potential surprises during units composition
 - 9.4 Limitations of systems of quantities
 - 9.4.1 Allowing irrational quantity combinations
 - 9.4.2 Quantities of dimension one
 - 9.5 Temperatures
 - 9.6 Structural types
- 10 Acknowledgements
- 11 References

§ 1 Revision history

§ 1.1 Changes since [P2981R0]

- Added The Guardian’s Fahrenheit issue to [Mismeasure for measure](#).
- Fuel consumption example extended in [Converting between quantities of the same kind](#).
- [\[Value Category Is Not Lifetime\]](#) reference added to [Preventing dangling references](#).
- Value-preserving type trait mentioned in [Lack of safe numeric types](#).
- [Non-negative quantities](#) rewritten.
- [Limitations of systems of quantities](#) added.
- [Temperatures](#) added.
- Some small editorial fixes.

§ 2 Introduction

One of the ways C++ can significantly improve the safety of applications being written by thousands of developers is by introducing a type-safe, well-tested, proven in production, standardized way to handle physical quantities and their units. The rationale is that people tend to have problems communicating or using proper units in code and daily life. Another benefit of adding strongly typed quantities and units to the standard is that it will allow catching some mistakes at compile-time instead of relying on runtime checks which might have spotty coverage.

This paper scopes on the safety aspects of introducing such a library to the C++ standard. In the following chapters, we will describe which industries are desperately looking for such standardized solutions, enumerate some failures and accidents caused by misinterpretation of quantities and units in human history, review all the features of such a library that improve the safety of our code, and also discuss potential safety issues that the library does not prevent against.

Note: This paper refers to practices used in the [\[mp-units\]](#) library and presents code examples using its interfaces. Those may not exactly reflect the final interface design that is going to be proposed in the follow-up papers. We are still doing some small fine-tuning to improve the library.

§ 3 Terms and definitions

This document consistently uses the official metrology vocabulary defined in the [\[ISO/IEC Guide 99\]](#) and [\[JCGM 200:2012\]](#).

§ 4 The future is here

Not that long ago, self-driving cars were a thing from SciFi movies. It was something so futuristic and hard to imagine that it only appeared in movies set in the very far future. Today, autonomous cars are becoming a reality on our streets even if they are not (yet) widely adopted.

It is no longer only the space industry or airline pilots that benefit from the autonomous operations of some machines. We live in a world where more and more ordinary people trust machines with their lives daily. The autonomous car is just one example which will affect our daily life. Medical devices, such as surgical robots and smart health care devices, are already a thing and will see wider adoption in the future. And there will be more machines with safety- or even life-critical tasks in the future. As a result, many more C++ engineers are expected to write life-critical software today than a few years ago.

Unfortunately, experience in this domain is hard to come by, and training alone will not solve the issue of mistakes caused by confusing units or quantities. Additionally, the C++ standard does not change fast enough to enforce a safe-by-construction code, which becomes even more critical if the code handling the physical computation is written by domain experts such as physicists that are not necessarily fluent in C++.

§ 5 Affected industries

When people think about industries that could use physical quantities and unit libraries, they think of a few companies related to aerospace, autonomous cars, or embedded industries. That is all true, but there are many other potential users for such a library.

Here is a list of some less obvious candidates:

- Manufacturing,
- maritime industry,
- freight transport,
- military,
- astronomy,
- 3D design,
- robotics,
- audio,
- medical devices,
- national laboratories,
- scientific institutions and universities,
- all kinds of navigation and charting,
- GUI frameworks,

— finance (including HFT).

As we can see, the range of domains for such a library is vast and not limited to applications involving specifically physical units. Any software that involves measurements, or operations on counts of some standard or domain-specific quantities, could benefit from a zero-cost abstraction for operating on quantity values and their units. The library also provides affine space abstractions, which may prove useful in many applications.

§ 6 Mismeasure for measure

Human history knows many expensive failures and accidents caused by mistakes in calculations involving different physical units. The most famous and probably the most expensive example in the software engineering domain is the Mars Climate Orbiter that in 1999 failed to enter Mars' orbit and crashed while entering its atmosphere [[Mars Orbiter](#)]. This is one of many examples here. People tend to confuse units quite often. We see similar errors occurring in various domains over the years:

- On October 12, 1492, Christopher Columbus unintentionally discovered the sea route from Europe to America because, during his travel preparations, he mixed the Arabic mile with a Roman mile, which led to the wrong estimation of the equator and his expected travel distance [[Columbus](#)].
- In 1628, a new warship, Vasa, accidentally had an asymmetrical hull (being thicker on the port side than the starboard side), which was one of the reasons for her sinking less than a mile into her maiden voyage, resulting in the death of 30 people on board. This asymmetry could have been caused by the use of different systems of measurement, as archaeologists have found four rulers used by the workers who built the ship. Two were calibrated in Swedish feet, which had 12 inches, while the other two measured Amsterdam feet, which had 11 inches [[Vasa](#)].
- Air Canada Flight 143 ran out of fuel on July 23, 1983, at an altitude of 41 000 feet (12 000 metres), midway through the flight because the fuel had been calculated in pounds instead of kilograms by the ground crew [[Gimli Glider](#)].
- The British rock band Black Sabbath, during its Born Again tour in 1983, ordered a replica of Stonehenge as props for the scene. Unfortunately, they had to leave them in the storage area because, while submitting the order, their manager wrote dimensions down in meters when he meant feet, and so the stones didn't fit the scene. "It cost a fortune to make, but there was not a building on Earth that you could fit it into" [[Stonehenge](#)].
- On April 15, 1999, Korean Air Cargo Flight 6316 crashed due to a miscommunication between pilots about the desired flight altitude [[Flight 6316](#)].
- In February 2001, the crew of the Moorpark College Zoo built an enclosure for Clarence the Tortoise with a weight of 250 pounds instead of 250 kilograms [[Clarence](#)].
- In December 2003, one of the roller coaster's cars at Tokyo Disneyland's Space Mountain attraction suddenly derailed due to a broken axle caused by confusion after upgrading the specification from imperial to metric units [[Disney](#)].
- During the construction of the Hochtiefbrücke bridge to connect the small German town of Laufenburg with Swiss Laufenburg, the construction team made a sign error that resulted in a discrepancy of 54 cm between the two outer ends of the bridge [[Hochtiefbrücke](#)].
- An American company sold a shipment of wild rice to a Japanese customer, quoting a price of 39 cents per pound, but the customer thought the quote was for 39 cents per kilogram [[Wild Rice](#)].
- On October 17, 2023, The Guardian published an article titled "Record Heat: Malawi swelters with temperatures nearly 68F above average" with many issues related to the affine space types and temperature units. Due to incorrect logic, probably during the translation of the article to the U.S. market, 20 °C above the average temperature was converted to 68 °F. The actual temperature increase was 32 °F, not 68 °F [[The Guardian](#)].
- A whole set of [[Medication dose errors](#)]...

§ 7 Common smells when there is no library for physical quantities

In this chapter, we are going to review typical safety issues related to physical quantities and units in the C++ code when a proper library is not used. Even though all the examples come from the Open Source projects, expensive revenue-generating production source code often is similar.

§ 7.1 The proliferation of `double`

It turns out that in the C++ software, most of our calculations in the physical quantities and units domain are handled with fundamental types like `double`. Code like below is a typical example here:

```
double GlidePolar::MacCreadyAltitude(double MCREADY,
                                     double Distance,
                                     const double Bearing,
                                     const double WindSpeed,
                                     const double WindBearing,
                                     double *BestCruiseTrack,
                                     double *VMacCready,
                                     const bool isFinalGlide,
                                     double *TimeToGo,
                                     const double AltitudeAboveTarget=1.0e6,
                                     const double cruise_efficiency=1.0,
                                     const double TaskAltDiff=-1.0e6);
```

[Original code here.](#)

There are several problems with such an approach: The abundance of `double` parameters makes it easy to accidentally switch values and there is no way of noticing such a mistake at compile-time. The code is not self-documenting in what units the parameters are expected. Is `Distance` in meters or kilometers? Is `WindSpeed` in meters per second or knots? Different code bases choose different ways to encode this information, which may be internally inconsistent. A strong type system would help answer these questions at the time the interface is written, and the compiler would verify it at compile-time.

§ 7.2 The proliferation of magic numbers

There are a lot of constants and conversion factors involved in the quantity equations. Source code responsible for such computations is often trashed with magic numbers:

```
double AirDensity(double hr, double temp, double abs_press)
{
    return (1/(287.06*(temp+273.15)))*(abs_press - 230.617 * hr *
    exp((17.5043*temp)/(241.2+temp)));
}
```

[Original code here.](#)

Apart from the obvious readability issues, such code is hard to maintain, and it needs a lot of domain knowledge on the developer's side. While it would be easy to replace these numbers with named constants, the question of which unit the constant is in remains. Is `287.06` in pounds per square inch (psi) or millibars (mbar)?

§ 7.3 The proliferation of conversion macros

The lack of automated unit conversions often results in handwritten conversion functions or macros that are spread everywhere among the code base:

```
#ifndef PI
static const double PI = (4*atan(1));
#endif
```

```

#define EARTH_DIAMETER    12733426.0    // Diameter of earth in meters
#define SQUARED_EARTH_DIAMETER    162140137697476.0    // Diameter of earth in meters
    (EARTH_DIAMETER*EARTH_DIAMETER)
#ifndef DEG_TO_RAD
#define DEG_TO_RAD    (PI / 180)
#define RAD_TO_DEG    (180 / PI)
#endif

#define NAUTICALMILESTOMETRES    (double)1851.96
#define KNOTSTOMETRESSECONDS    (double)0.5144

#define TOKNOTS    (double)1.944
#define TOFEETPERMINUTE    (double)196.9
#define TOMPH    (double)2.237
#define TOKPH    (double)3.6

// meters to.. conversion
#define TONAUTICALMILES    (1.0 / 1852.0)
#define TOMILES    (1.0 / 1609.344)
#define TOKILOMETER    (0.001)
#define TOFEET    (1.0 / 0.3048)
#define TOMETER    (1.0)

```

[Original code here.](#)

Again, the question of which unit the constant is in remains. Without looking at the code, it is impossible to tell from which unit TOMETER converts. Also, macros have the problem that they are not scoped to a namespace and thus can easily clash with other macros or functions, especially if they have such common names like PI or RAD_TO_DEG. A quick search through open source C++ code bases reveals that, for example, the RAD_TO_DEG macro is defined in a multitude of different ways – sometimes even within the same repository:

```

#define RAD_TO_DEG    (180 / PI)
#define RAD_TO_DEG    57.2957795131
#define RAD_TO_DEG    ( radians ) ((radians ) * 180.0 / M_PI)
#define RAD_TO_DEG    57.2957805f
...

```

[Example search across multiple repositories](#)

[Multiple redefinitions in the same repository](#)

Another safety issue occurring here is the fact that macro values can be deliberately tainted by compiler settings at built time and can acquire values that are not present in the source code. Human reviews won't catch such issues.

Also, most of the macros do not follow best practices. Often, necessary parentheses are missing, processing in a preprocessor ends up with redundant casts, or some compile-time constants use too many digits for a value to be exact for a specific type (e.g. float).

§ 7.4 Lack of consistency

If we not only lack strong types to isolate the abstractions from each other, but also lack discipline to keep our code consistent, we end up in an awful place:

```

void DistanceBearing(double lat1, double lon1,
                    double lat2, double lon2,
                    double *Distance, double *Bearing);

double DoubleDistance(double lat1, double lon1,
                     double lat2, double lon2,
                     double lat3, double lon3);

void FindLatitudeLongitude(double Lat, double Lon,

```

```

        double Bearing, double Distance,
        double *lat_out, double *lon_out);

double CrossTrackError(double lon1, double lat1,
                      double lon2, double lat2,
                      double lon3, double lat3,
                      double *lon4, double *lat4);

double ProjectedDistance(double lon1, double lat1,
                       double lon2, double lat2,
                       double lon3, double lat3,
                       double *xtd, double *crs);

```

[Original code here.](#)

Users can easily make errors if the interface designers are not consistent in ordering parameters. It is really hard to remember which function takes latitude or Bearing first and when a latitude or Distance is in the front.

§ 7.5 Lack of a conceptual framework

The previous points mean that the fundamental types can't be leveraged to model the different concepts of quantities and units frameworks. There is no shared vocabulary between different libraries. User-facing APIs use ad-hoc conventions. Even internal interfaces are inconsistent between themselves.

Arithmetic types such as `int` and `double` are used to model different concepts. They are used to represent any abstraction (be it a magnitude, difference, point, or kind) of any quantity type of any unit. These are weak types that make up weakly-typed interfaces. The resulting interfaces and implementations built with these types easily allow mixing up parameters and using operations that are not part of the represented quantity.

§ 8 Safety features

This chapter describes the features that enforce safety in our code bases. It starts with obvious things, but then it moves to probably less known benefits of using physical quantities and units libraries. This chapter also serves as a proof that it is not easy to implement such a library correctly, and that there are many cases where the lack of experience or time for the development of such a utility may easily lead to safety issues as well.

Before we go through all the features, it is essential to note that they do not introduce any runtime overhead over the raw unsafe code doing the same thing. This is a massive benefit of C++ compared to other programming languages (e.g., Python, Java, etc.).

§ 8.1 Unit conversions

The first thing that comes to our mind when discussing the safety of such libraries is automated unit conversions between values of the same physical quantity. This is probably the most important subject here. We learned about its huge benefits long ago thanks to the `std::chrono::duration` that made conversions of time durations error-proof.

Unit conversions are typically performed either via a converting constructor or a dedicated conversion function:

```

auto q1 = 5 * km;
std::cout << q1.in(m) << '\n';
quantity<si::metre, int> q2 = q1;

```

Such a feature benefits from the fact that the library knows about the magnitudes of the source and destination units at compile-time, and may use that information to calculate and apply a conversion factor automatically for the user.

In `std::chrono::duration`, the magnitude of a unit is always expressed with `std::ratio`. This is not enough for a general-purpose physical units library. Some of the derived units have huge or tiny ratios. The difference from

the base units is so huge that it cannot be expressed with `std::ratio`, which is implemented in terms of `std::intmax_t`. This makes it impossible to define units like electronvolt (eV), where $1 \text{ eV} = 1.602176634 \times 10^{-19} \text{ J}$, or Dalton (Da), where $1 \text{ Da} = 1.660539040(20) \times 10^{-27} \text{ kg}$. Moreover, some conversions, such as radian to a degree, require a conversion factor based on an irrational number like π .

§ 8.2 Preventing truncation of data

The second safety feature of such libraries is preventing accidental truncation of a quantity value. If we try the operations above with swapped units, both conversions should fail to compile:

```
auto q1 = 5 * m;
std::cout << q1.in(km) << '\n';           // Compile-time error
quantity<si::kilo<si::metre>, int> q2 = q1; // Compile-time error
```

We can't preserve the value of a source quantity when we convert it to one with a unit of a lower resolution while dealing with an integral representation type for a quantity. In the example above, converting 5 meters would result in 0 kilometers if internal conversion is performed using regular integer arithmetic.

While this could be a valid behavior, the problem arises when the user expects to be able to convert the quantity back to the original unit without loss of information. So the library should prevent such conversions from happening implicitly; [\[mp-units\]](#) offers the named cast `value_cast` for these conversions marked as unsafe.

To make the above conversions compile, we could use a floating-point representation type:

```
auto q1 = 5. * m; // source quantity uses `double` as a representation type
std::cout << q1.in(km) << '\n';
quantity<si::kilo<si::metre>> q2 = q1;
```

or:

```
auto q1 = 5 * m; // source quantity uses `int` as a representation type
std::cout << value_cast<double>(q1).in(km) << '\n';
quantity<si::kilo<si::metre>> q2 = q1; // `double` by default
```

The [\[mp-units\]](#) library follows `std::chrono::duration` logic and treats floating-point types as value-preserving.

Another possibility would be to force such a truncating conversion explicitly from the code:

```
auto q1 = 5 * m; // source quantity uses `int` as a representation type
std::cout << q1.force_in(km) << '\n';
quantity<si::kilo<si::metre>, int> q2 = value_cast<km>(q1);
```

The code above makes it clear that “something bad” may happen here if we are not extra careful.

Another case for truncation happens when we assign a quantity with a floating-point representation type to the one using an integral representation type for its value:

```
auto q1 = 2.5 * m;
quantity<si::metre, int> q2 = q1;
```

Such an operation should fail to compile as well. Again, to force such a truncation, we have to be explicit in the code:

```
auto q1 = 2.5 * m;
quantity<si::metre, int> q2 = value_cast<int>(q1);
```

As we can see, it is essential not to allow such truncating conversions to happen implicitly, and a good physical quantities and units library should fail at compile-time in case an user makes such a mistake.

§ 8.3 The affine space

The affine space has two types of entities:

- point - a position specified with coordinate values (i.e., location, address, etc.)
- vector - the difference between two points (i.e., shift, offset, displacement, duration, etc.)

One can do a limited set of operations in affine space on points and vectors. This greatly helps to prevent quantity equations that do not have physical sense.

People often think that affine space is needed only to model temperatures and maybe time points (following the `std::chrono::time_point` example). Still, the applicability of this concept is much wider.

For example, if we would like to model a Mount Everest climbing expedition, we would deal with two kinds of altitude-related entities. The first would be absolute altitudes above the mean sea level (points) like base camp altitude, mount peak altitude, etc. The second one would be the heights of daily climbs (vectors). Although it makes physical sense to add heights of daily climbs, there is no sense in adding altitudes. What does adding the altitude of a base camp and the mountain peak mean after all?

Modeling such affine space entities with the `quantity` (vector) and `quantity_point` (point) class templates improves the overall project's safety by only providing the operators defined by the concepts.

§ 8.4 `explicit` is not explicit enough

Consider the following structure and a code using it:

```
struct X {  
    std::vector<std::chrono::milliseconds> vec;  
    // ...  
};
```

```
X x;  
x.vec.emplace_back(42);
```

Everything works fine for years until, at some point, someone changes the structure to:

```
struct X {  
    std::vector<std::chrono::microseconds> vec;  
    // ...  
};
```

The code continues to compile fine, but all the calculations are now off by orders of magnitude. This is why a good physical quantities and units library should not provide an explicit quantity constructor taking a raw value.

To solve this issue, a quantity in the [mp-units] library always requires information about both a number and a unit during construction:

```
struct X {  
    std::vector<quantity<si::milli<si::seconds>>> vec;  
    // ...  
};
```

```
X x;  
x.vec.emplace_back(42);           // Compile-time error  
x.vec.emplace_back(42 * ms);     // OK
```

For consistency and to prevent similar safety issues, the `quantity_point` in the [mp-units] library can't be created from a standalone value of a quantity (contrary to the `std::chrono::time_point` design). Such a point has to always be associated with an explicit origin:

```
quantity_point qp1 = mean_sea_level + 42 * m;  
quantity_point qp2 = si::ice_point + 21 * deg_C;
```

§ 8.5 Obtaining the numerical value of a quantity

Continuing our previous example, let’s assume that we have an underlying “legacy” API that requires us to pass a raw numerical value of a quantity and that we do the following to use it:

```
void legacy_func(std::int64_t seconds);
```

```
X x;  
x.vec.emplace_back(42s);  
legacy_func(x.vec[0].count());
```

The preceding code is incorrect. Even though the duration stores a quantity equal to 42 s, it is not stored in seconds (it’s either microseconds or milliseconds, depending on which of the interfaces from the previous chapter is the current one). Such issues can be prevented with the usage of `std::chrono::duration_cast`:

```
legacy_func(duration_cast<seconds>(x.vec[0]).count());
```

However, users often forget about this step, especially when, at the moment of writing such code, the duration stores the underlying raw value in the expected unit. But as we know, the interface can be refactored at any point to use a different unit, and the code using an underlying numerical value without the usage of an explicit cast will become incorrect.

To prevent such safety issues, the `[mp-units]` library exposes only the interface that returns a quantity numerical value in the required unit to ensure that no data truncation happens:

```
X x;  
x.vec.emplace_back(42 * s);  
legacy_func(x.vec[0].numerical_value_in(si::second));
```

or in case we are fine with data truncation:

```
X x;  
x.vec.emplace_back(42 * s);  
legacy_func(x.vec[0].force_numerical_value_in(si::second));
```

As the above member functions may need to do a conversion to provide a value in the expected unit, their results are prvalues.

§ 8.6 Preventing dangling references

Besides returning prvalues, sometimes users need to get an actual reference to the underlying numerical value stored in a quantity. For those cases, the `[mp-units]` library exposes `quantity::numerical_value_ref_in(Unit)` that participates in overload resolution only:

- for lvalues (rvalue reference qualified overload is explicitly deleted),
- when the provided `Unit` has the same magnitude as the one currently used by the quantity.

The first condition above aims to limit the possibility of dangling references. We want to increase the chances that the reference/pointer provided to an underlying API remains valid for the time of its usage. (We’re much less concerned about performance aspects for a quantity here, as we expect the majority (if not all) of representation types to be cheap to copy.)

That said, we acknowledge that this approach to preventing dangling references conflates value category with lifetime. While it may prevent the majority of dangling references, it also admits both false positives and false negatives, as explained in [\[Value Category Is Not Lifetime\]](#). We want to highlight this dilemma for the committee’s consideration.

In case we do decide to keep this policy of deleting rvalue overloads, here’s an example of code that it would prevent from compiling.

```
void legacy_func(const int& seconds);

legacy_func((4 * s + 2 * s).numerical_value_ref_in(si::second)); // Compile-time error
```

The [mp-units] library goes one step further, by implementing all compound assignments, pre-increment, and pre-decrement operators as non-member functions that preserve the initial value category. Thanks to that, the following will also not compile:

```
quantity<si::second, int> get_duration();

legacy_func((4 * s += 2 * s).numerical_value_ref_in(si::second)); // Compile-time error
legacy_func(++get_duration()).numerical_value_ref_in(si::second)); // Compile-time error
```

The second condition above enables the usage of various equivalent units. For example, J is equivalent to N * m, and kg * m2 / s2. As those have the same magnitude, it does not matter exactly which one is being used here, as the same numerical value should be returned for all of them.

```
void legacy_func(const int& joules);

quantity q1 = 42 * J;
quantity q2 = 42 * N * (2 * m);
quantity q3 = 42 * kJ;
legacy_func(q1.numerical_value_ref_in(si::joule)); // OK
legacy_func(q2.numerical_value_ref_in(si::joule)); // OK
legacy_func(q3.numerical_value_ref_in(si::joule)); // Compile-time error
```

Here are a few examples provided by our users where enabling a quantity type to return a reference to its underlying numerical value is required:

— Interoperability with the Dear ImGui:

```
IMGUI_API bool DragInt(const char* label, int* v,
                      float v_speed = 1.0f, int v_min = 0, int v_max = 0,
                      const char* format = "%d", ImGuiSliderFlags flags = 0); // If
                      v_min >= v_max we have no bound

ImGui::DragInt("Frames", &frame_frequency.value.numerical_value_ref_in(Hz), 1,
              frame_frequency.min.numerical_value_in(Hz),
              frame_frequency.max.numerical_value_in(Hz),
              "%d Hz");
```

— Obtaining a temperature via the C library:

```
// read_temperature is in the BSP defined as
void read_temperature(float* temp_in_celsius);

using Celsius_point = quantity_point<isq::Celsius_temperature[deg_C], si::ice_point,
float>;

Celsius_point temp;
read_temperature(&temp.quantity_ref_from(si::ice_point).numerical_value_ref_in(si::degree_Celsius));
```

As we can see in the second example, quantity_point also provides an lvalue-ref-qualified quantity_ref_from(PointOrigin) member function that returns a reference to its stored quantity. Also, for reasons similar to the ones described in the previous chapter, this function requires that the argument provided by the user is the same as the origin of a quantity point.

§ 8.7 Quantity kinds

What should be the result of the following quantity equation?

```
auto res = 1 * Hz + 1 * Bq + 1 * Bd;
```

We have checked a few leading libraries on the market, and here are the results:

- [Boost.Units] claims the answer to be 2 Hz (bauds are not supported by it, so we removed it from the equation),
- [nholthaus/units] claims it is 2 s^{-1} (no support for bauds as well),
- [Pint] library in Python claims that the result is 3.0 Hz,
- [JSR 385] library in Java throws an exception saying that we can't add those quantities.

Now let's check what [ISO/IEC Guide 99] says about quantity kinds:

- Quantities may be grouped together into categories of quantities that are **mutually comparable**
- Mutually comparable quantities are called **quantities of the same kind**
- Two or more quantities **cannot be added or subtracted unless they belong to the same category of mutually comparable quantities**
- Quantities of the **same kind** within a given system of quantities **have the same quantity dimension**
- Quantities of the **same dimension are not necessarily of the same kind**

[ISO 80000] also explicitly notes:

Measurement units of quantities of the same quantity dimension may be designated by the same name and symbol even when the quantities are not of the same kind. For example, joule per kelvin and J/K are respectively the name and symbol of both a measurement unit of heat capacity and a measurement unit of entropy, which are generally not considered to be quantities of the same kind. **However, in some cases special measurement unit names are restricted to be used with quantities of specific kind only.** For example, the measurement unit 'second to the power minus one' ($1/\text{s}$) is called hertz (Hz) when used for frequencies and becquerel (Bq) when used for activities of radionuclides. As another example, the joule (J) is used as a unit of energy, but never as a unit of moment of force, i.e. the newton metre ($\text{N} \cdot \text{m}$).

To summarize the above, [ISO 80000] explicitly states that frequency is measured in Hz and activity is measured in Bq, which are quantities of different kinds. As such, they should not be able to be compared, added, or subtracted. So, the only library from the above that was correct was [JSR 385]. The rest of them are wrong to allow such operations. Doing so may lead to vulnerable safety issues when two unrelated quantities of the same dimension are accidentally added or assigned to each other.

The reason for most of the libraries on the market to be wrong in this field is the fact that their quantities are implemented only in terms of the concept of dimension. However, we've just learned that a dimension is not enough to express a quantity type.

The [mp-units] library goes beyond that and properly models quantity kinds. We believe that it is a significant feature that improves the safety of the library, and that is why we also plan to propose quantity kinds for standardization as mentioned in [P2980R0].

§ 8.8 Various quantities of the same kind

Proper modeling of distinct kinds for quantities of the same dimension is often not enough from the safety point of view. Most of the libraries allow us to write the following code in the type-safe way:

```
quantity<isq::speed[m/s]> avg_speed(quantity<isq::length[m]> l, quantity<isq::time[s]> t)
{
    return l / t;
}
```

However, they fail when we need to model an abstraction using more than one quantity of the same kind:

```
class Box {
    quantity<isq::area[m2]> base_;
```

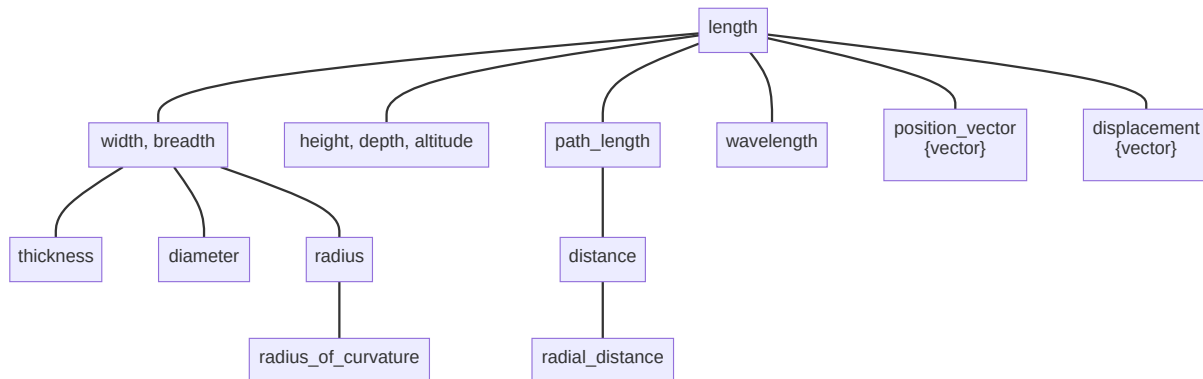
```

    quantity<isq::length[m]> height_;
public:
    Box(quantity<isq::length[m]> l, quantity<isq::length[m]> w, quantity<isq::length[m]> h)
        : base_(l * w), height_(h)
    {}
    // ...
};

```

This does not provide strongly typed interfaces anymore.

Again, it turns out that [ISO 80000](#) has an answer. This specification standardizes hundreds of quantities, many of which are of the same kind. For example, for quantities of the kind length, it provides the following:



As we can see, various quantities of the same kind are not a flat set. They form a hierarchy tree which influences

- conversion rules, and
- the quantity type being the result of adding or subtracting different quantities of the same kind.

The [\[mp-units\]](#) library is probably the first one on the market (in any programming language) that models such abstractions.

§ 8.8.1 Converting between quantities of the same kind

Quantity conversion rules can be defined based on the same hierarchy of quantities of kind length.

1. Implicit conversions

- Every width is a length.
- Every radius is a width.

```

static_assert(implicitly_convertible(isq::width, isq::length));
static_assert(implicitly_convertible(isq::radius, isq::length));
static_assert(implicitly_convertible(isq::radius, isq::width));

```

In the [\[mp-units\]](#) library, implicit conversions are allowed on copy-initialization:

```

void foo(quantity<isq::length[m]> q);

quantity<isq::width[m]> q1 = 42 * m;
quantity<isq::length[m]> q2 = q1; // implicit quantity conversion
foo(q1);                          // implicit quantity conversion

```

2. Explicit conversions

- Not every length is a width.
- Not every width is a radius.

```

static_assert(!implicitly_convertible(isq::length, isq::width));
static_assert(!implicitly_convertible(isq::length, isq::radius));

```

```
static_assert(!implicitly_convertible(isq::width, isq::radius));
static_assert(implicitly_convertible(isq::length, isq::width));
static_assert(implicitly_convertible(isq::length, isq::radius));
static_assert(implicitly_convertible(isq::width, isq::radius));
```

In the [mp-units] library, explicit conversions are forced by passing the quantity to a call operator of a quantity_spec type:

```
quantity<isq::length<m>> q1 = 42 * m;
quantity<isq::height<m>> q2 = isq::height(q1); // explicit quantity conversion
```

3. Explicit casts

- height is never a width, and vice versa.
- Both height and width are quantities of kind length.

```
static_assert(!implicitly_convertible(isq::height, isq::width));
static_assert(!explicitly_convertible(isq::height, isq::width));
static_assert(castable(isq::height, isq::width));
```

In the [mp-units] library, explicit casts are forced with a dedicated quantity_cast function:

```
quantity<isq::width<m>> q1 = 42 * m;
quantity<isq::height<m>> q2 = quantity_cast<isq::height>(q1); // explicit quantity cast
```

4. No conversion

- time has nothing in common with length.

```
static_assert(!implicitly_convertible(isq::time, isq::length));
static_assert(!explicitly_convertible(isq::time, isq::length));
static_assert(!castable(isq::time, isq::length));
```

In the [mp-units] library, even the explicit casts will not force such a conversion:

```
void foo(quantity<isq::length[m]>);

foo(quantity_cast<isq::length>(42 * s)); // Compile-time error
```

With the above rules, one can write the following short application to calculate a fuel consumption:

```
inline constexpr struct fuel_volume : quantity_spec<isq::volume> {} fuel_volume;
inline constexpr struct fuel_consumption : quantity_spec<fuel_volume / isq::distance> {}
    fuel_consumption;

const quantity fuel = fuel_volume(40. * l);
const quantity distance = isq::distance(550. * km);
const quantity<fuel_consumption[l / (mag<100> * km)]> q = fuel / distance;
std::cout << "Fuel consumption: " << q << "\n";
```

The above code prints:

Fuel consumption: 7.27273×10^{-2} l/km

Please note that, despite the dimensions of fuel_consumption and isq::area being the same (L²), the constructor of a quantity q below will fail to compile when we pass an argument being the quantity of area:

```
static_assert(fuel_consumption.dimension == isq::area.dimension);

const quantity<isq::area[m2]> football_field = isq::length(105 * m) * isq::width(68 * m);
const quantity<fuel_consumption[l / (mag<100> * km)]> q2 = football_field; // Compile-time
    error
const quantity q3 = q + football_field; // Compile-time
    error
if (q == football_field) { // Compile-time
    error
```

```
// ...
}
```

§ 8.8.2 Comparing, adding, and subtracting quantities of the same kind

[ISO/IEC Guide 99] explicitly states that width and height are quantities of the same kind and as such they

- are mutually comparable, and
- can be added and subtracted.

If we take the above for granted, the only reasonable result of $1 * \text{width} + 1 * \text{height}$ is $2 * \text{length}$, where the result of length is known as a common quantity type. A result of such an equation is always the first common node in a hierarchy tree of the same kind. For example:

```
static_assert(common_quantity_spec(isq::width, isq::height) == isq::length);
static_assert(common_quantity_spec(isq::thickness, isq::radius) == isq::width);
static_assert(common_quantity_spec(isq::distance, isq::path_length) == isq::path_length);

quantity q = isq::thickness(1 * m) + isq::radius(1 * m);
static_assert(q.quantity_spec == isq::width);
```

One could argue that allowing to add or compare quantities of height and width might be a safety issue, but we need to be consistent with the requirements of [ISO 80000]. Moreover, from our experience, disallowing such operations and requiring an explicit cast to a common quantity in every single place makes the code so cluttered with casts that it nearly renders the library unusable.

Fortunately, the above-mentioned conversion rules make the code safe by construction anyway. Let's analyze the following example:

```
inline constexpr struct horizontal_length : quantity_spec<isq::length> {} horizontal_length;

namespace christmas {

struct gift {
    quantity<horizontal_length[m]> length;
    quantity<isq::width[m]> width;
    quantity<isq::height[m]> height;
};

std::array<quantity<isq::length[m]>, 2> gift_wrapping_paper_size(const gift& g)
{
    const auto dim1 = 2 * g.width + 2 * g.height + 0.5 * g.width;
    const auto dim2 = g.length + 2 * 0.75 * g.height;
    return { dim1, dim2 };
}

} // namespace christmas

int main()
{
    const christmas::gift lego = { horizontal_length(40 * cm), isq::width(30 * cm),
                                   isq::height(15 * cm) };
    auto paper = christmas::gift_wrapping_paper_size(lego);

    std::cout << "Paper needed to pack a lego box:\n";
    std::cout << "- " << paper[0] << " x " << paper[1] << "\n"; // - 1.05 m X 0.625 m
    std::cout << "- area = " << paper[0] * paper[1] << "\n"; // - area = 0.65625 m²
}
```

In the beginning, we introduce a custom quantity `horizontal_length` of a kind `length`, which then, together with `isq::width` and `isq::height`, are used to define the dimensions of a Christmas gift. Next, we provide a function that calculates the dimensions of a gift wrapping paper with some wraparound. The result of both those

expressions is a quantity of `isq::length`, as this is the closest common quantity for the arguments used in this quantity equation.

Regarding safety, it is important to mention here, that thanks to the conversion rules provided above, it would be impossible to accidentally do the following:

```
void foo(quantity<horizontal_length[m]> q);

quantity<isq::width[m]> q1 = dim1; // Compile-time error
quantity<isq::height[m]> q2{dim1}; // Compile-time error
foo(dim1);                        // Compile-time error
```

The reason of compilation errors above is the fact that `isq::length` is not implicitly convertible to the quantities defined based on it. To make the above code compile, an explicit conversion of a quantity type is needed:

```
void foo(quantity<horizontal_length[m]> q);

quantity<isq::width[m]> q1 = isq::width(dim1);
quantity<isq::height[m]> q2{isq::height(dim1)};
foo(horizontal_length(dim1));
```

To summarize, rules for addition, subtraction, and comparison of quantities improve the library usability, while the conversion rules enhance the safety of the library compared to the libraries that do not model quantity kinds.

§ 8.8.3 Modeling a quantity kind

In the physical units library, we also need an abstraction describing an entire family of quantities of the same kind. Such quantities have not only the same dimension but also can be expressed in the same units.

To annotate a quantity to represent its kind we introduced the `kind_of<>` specifier. For example, to express any quantity of length, we need to type `kind_of<isq::length>`. Such an entity behaves as any quantity of its kind. This means that it is implicitly convertible to any quantity in a hierarchy tree.

```
static_assert(!implicitly_convertible(isq::length, isq::height));
static_assert(implicitly_convertible(kind_of<isq::length>, isq::height));
```

Additionally, the result of operations on quantity kinds is also a quantity kind:

```
static_assert(same_type<kind_of<isq::length> / kind_of<isq::time>, kind_of<isq::length / isq::time>>);
```

However, if at least one equation's operand is not a kind, the result becomes a “strong” quantity where all the kinds are converted to the hierarchy tree's root quantities:

```
static_assert(!same_type<kind_of<isq::length> / isq::time, kind_of<isq::length / isq::time>>);
static_assert(same_type<kind_of<isq::length> / isq::time, isq::length / isq::time>);
```

§ 8.8.4 Restricting units to specific quantity kinds

By default, units can be used to measure any kind of quantity with the same dimension. However, as we have mentioned above, some units (e.g., Hz, Bq) are constrained to be used only with a specific kind. Also, base units of the SI are meant to measure all of the quantities of their kinds. To model this, in the [\[mp-units\]](#) library, we do the following:

```
// base units
inline constexpr struct metre : named_unit<"m", kind_of<isq::length>> {} metre;
inline constexpr struct second : named_unit<"s", kind_of<isq::time>> {} second;

// derived units
inline constexpr struct hertz : named_unit<"Hz", 1 / second, kind_of<isq::frequency>> {} hertz;
```



```
inline constexpr struct becquerel : named_unit<"Bq", 1 / second, kind_of<isq::activity>> {}
    becquerel;
inline constexpr struct baud : named_unit<"Bd", 1 / si::second,
    kind_of<iec80000::modulation_rate>> {} baud;
```

This means that every time we type `42 * m`, we create a quantity of a kind length with the length dimension. Such a quantity can be added, subtracted, or compared to any other quantity of the same kind. Moreover, it is implicitly convertible to any quantity of its kind. Again, this could be considered a safety issue as one could type:

```
const christmas::gift lego = { 40 * cm, 30 * cm, 15 * cm };
auto paper = christmas::gift_wrapping_paper_size(lego);
```

The above code compiles fine without the need to force specific quantity types during construction. This is another tradeoff we have to do here in order to improve the usability. Otherwise, we would need to type the following every single time we want to initialize an array or aggregate:

```
const quantity<isq::position_vector[m], int> measurements[] = { isq::position_vector(30'160 *
    m),
                                                                isq::position_vector(30'365 *
    m),
                                                                isq::position_vector(30'890 *
    m),
                                                                isq::position_vector(31'050 *
    m),
                                                                isq::position_vector(31'785 *
    m),
                                                                isq::position_vector(32'215 *
    m),
                                                                isq::position_vector(33'130 *
    m),
                                                                isq::position_vector(34'510 *
    m),
                                                                isq::position_vector(36'010 *
    m),
                                                                isq::position_vector(37'265 *
    m) };
```

As we can see above, it would be really inconvenient. With the current rules, we type:

```
const quantity<isq::position_vector[m], int> measurements[] = { 30'160 * m, 30'365 * m,
    30'890 * m, 31'050 * m,
                                                                31'785 * m, 32'215 * m,
    33'130 * m, 34'510 * m,
                                                                36'010 * m, 37'265 * m };
```

which is more user-friendly.

Having such two options also gives users a choice. When we use different quantities of the same kind in a project (e.g., radius, wavelength, altitude), we should probably reach for strongly-typed quantities to bring additional safety for those cases. Otherwise, we can just use the simple mode for the remaining quantities. We can easily mix simple and strongly-typed quantities in our projects, and the library will do its best to protect us based on the information provided.

§ 8.9 Non-negative quantities

Some quantity types are defined by [\[ISO 80000\]](#) as explicitly non-negative. Those include quantities like width, thickness, diameter, and radius. However, it turns out that it is possible to have negative values of quantities defined as non-negative. For example, `-1 * isq::diameter[mm]` could represent a change in the diameter of some object. Also, a subtraction `4 * width[mm] - 6 * width[mm]` results in a negative value as the width of the second argument is larger than the first one.

Non-negative quantities are not limited to those explicitly stated as being non-negative in the [\[ISO 80000\]](#). Some quantities are implicitly non-negative from their definition. The most obvious example here might be scalar

quantities specified as magnitudes of a vector quantity. For example, speed is defined as the magnitude of velocity. Again, $-1 * \text{speed}[\text{m/s}]$ could represent a change in average speed between two measurements.

This means that enforcing such constraints for quantity types might be impossible as those typically are used to represent a difference between two states or measurements. However, we could apply such constraints to quantity points, which, by definition, describe the absolute quantity values. For example, when height is the measure of an object, a negative value is physically meaningless.

Such logic errors could be detected at runtime with contracts or some other preconditions or invariants checks.

§ 8.10 Vector and tensor quantities

While talking about physical quantities and units libraries, everyone expects that the library will protect (preferably at compile-time) from accidentally replacing multiplication with division operations or vice versa. Everyone knows and expects that the multiplication of length and time should not result in speed. It does not mean that such a quantity equation is invalid. It just results in a quantity of a different type.

If we expect the above protection for scalar quantities, we should also strive to provide similar guarantees for vector and tensor quantities. First, the multiplication or division of two vectors or tensors is not even mathematically defined. Such operations should be impossible on quantities using vector or tensor representation types.

While multiplication and division are with scalars, the dot and cross products are for vector quantities. The result of the first one is a scalar. The second one results in a vector perpendicular to both vectors passed as arguments. A good physical quantities and units library should protect the user from making such an error of accidentally replacing those operations.

Vector and tensor quantities can be implemented in two ways:

1. Encapsulating multiple quantities into a homogeneous vector or tensor representation type

This solution is the most common in the C++ market. It requires the quantities library to provide only basic arithmetic operations (addition, subtraction, multiplication, and division) which are being used to calculate the result of linear algebra math. However, this solution can't provide any compile-time safety described above, and will also crash when someone passes a proper vector and tensor representation type to a quantity, expecting it to work.

2. Encapsulating a vector or tensor as a representation type of a quantity

This provides all the required type safety, but requires the library to implement more operations on quantities and properly constrain them so they are selectively enabled when needed. Besides [\[mp-units\]](#), the only library that supports such an approach is [\[Pint\]](#). Such a solution requires the following operations to be exposed for quantity types (note that character refers to the algebraic structure of either scalar, vector and tensor):

- $a + b$ - addition where both arguments should be of the same quantity kind and character
- $a - b$ - subtraction where both arguments should be of the same quantity kind and character
- $a \% b$ - modulo where both arguments should be of the same quantity kind and character
- $a * b$ - multiplication where one of the arguments has to be a scalar
- a / b - division where the divisor has to be scalar
- $a \cdot b$ - dot product of two vectors
- $a \times b$ - cross product of two vectors
- $|a|$ - magnitude of a vector

- $a \otimes b$ - tensor product of two vectors or tensors
- $a \cdot b$ - inner product of two tensors
- $a \cdot b$ - inner product of tensor and vector
- $a : b$ - scalar product of two tensors

Additionally, the [\[mp-units\]](#) library knows the expected quantity character, which is provided (implicitly or explicitly) in the definition of each quantity type. Thanks to that, it prevents the user, for example, from providing a scalar representation type for force or a vector representation for power quantities.

```
QuantityOf<isq::velocity> q1 = 60 * km / h;           // Compile-time error
QuantityOf<isq::velocity> q2 = la_vector{0, 0, -60} * km / h; // OK
QuantityOf<isq::force> q3 = 80 * kg * (10 * m / s2);    // Compile-time error
QuantityOf<isq::force> q4 = 80 * kg * (la_vector{0, 0, -10} * m / s2); // OK
QuantityOf<isq::power> q5 = q2 * q4;                   // Compile-time error
QuantityOf<isq::power> q5 = dot(q2, q4);                // OK
```

Note: $q1$ and $q3$ can be permitted to compile by explicitly specializing the `is_vector<T>` trait for the representation type.

As we can see above, such features additionally improves the compile-time safety of the library by ensuring that quantities are created with proper quantity equations and are using correct representation types.

§ 9 Safety pitfalls

§ 9.1 Integer division

The physical units library can't do any runtime branching logic for the division operator. All logic has to be done at compile-time when the actual values are not known, and the quantity types can't change at runtime.

If we expect $120 \cdot \text{km} / (2 \cdot \text{h})$ to return $60 \cdot \text{km} / \text{h}$, we have to agree with the fact that $5 \cdot \text{km} / (24 \cdot \text{h})$ returns $0 \cdot \text{km}/\text{h}$. We can't do a range check at runtime to dynamically adjust scales and types based on the values of provided function arguments.

The same applies to:

```
static_assert(5 * h / (120 * min) == 0 * one);
```

This is why floating-point representation types are recommended as a default to store the numerical value of a quantity. Some popular physical units libraries even [forbid integer division at all](#).

The problem is similar to the one described in the section about accidental truncation of values through conversion. While the consequent use of floating-point representation types may be a good idea, it is not always possible. Especially in close-to-the-metal applications and small embedded systems, the use of floating-point types is sometimes not an option, either for performance reasons or lack of hardware support. Having different operators for safe floating-point operations and unsafe integer operations would hurt generic programming. As such, users should instead use safer representation types.

§ 9.2 Lack of safe numeric types

Integers can overflow on arithmetics. This has already caused some expensive failures in engineering [\[Ariane flight V88\]](#).

Integers can also be truncated during assignment to a narrower type.

Floating-point types may lose precision during assignment to a narrower type. Conversion from `std::int64_t` to `double` may also lose precision.

If we had safe numeric types in the C++ standard library, they could easily be used as a quantity representation type in the physical quantities and units library, which would address these safety concerns.

Also, having a type trait informing if a conversion from one type to another is value-preserving would help to address some of the issues mentioned above.

§ 9.3 Potential surprises during units composition

One of the most essential requirements for a good physical quantities and units library is to implement units in such a way that they compose. With that, one can easily create any derived unit using a simple unit equation on other base or derived units. For example:

```
constexpr Unit auto kmph = km / h;
```

We can also easily obtain a quantity with:

```
quantity q = 60 * km / h;
```

Such a solution is an industry standard and is implemented not only in [mp-units], but also is available for many years now in both [Boost.Units] and [Pint].

We believe that is the correct thing to do. However, we want to make it straight in this paper that some potential issues are associated with such a syntax. Inexperienced users are often surprised by the results of the following expression:

```
quantity q = 60 * km / 2 * h;
```

This looks like like 30 km/h, right? But it is not. Thanks to the order of operations, it results in 30 km·h. In case we want to divide 60 km by 2 h, parentheses are needed:

```
quantity q = 60 * km / (2 * h);
```

Another surprising issue may result from the following code:

```
template<typename T>
auto make_length(T v) { return v * si::metre; }

auto v = 42;
quantity q = make_length(v);
```

This might look like a good idea, but let's consider what would happen if the user provided a quantity as input:

```
auto v = 42 * m;
quantity q = make_length(v);
```

The above function call will result in a quantity of area instead of the expected quantity of length.

The issues mentioned above could be turned into compilation errors by disallowing multiplying or dividing a quantity by a unit. The [mp-units] library initially provided such an approach, but with time, we decided this to not be user-friendly. Forcing the user to put the parenthesis around all derived units in quantity equations like the one below, was too verbose and confusing:

```
quantity q = 60 * (km / h);
```

It is important to notice that the problems mentioned above will always surface with a compile-time error at some point in the user's code when they assign the resulting quantity to one with an explicitly provided quantity type.

Below, we provide a few examples that correctly detect such issues at compile-time:

```

quantity<si::kilo<si::metre> / non_si::hour, int> q1 = 60 * km / 2 * h;           //
    Compile-time error
quantity<isq::speed[si::kilo<si::metre> / non_si::hour], int> q2 = 60 * km / 2 * h; //
    Compile-time error
QuantityOf<isq::speed> auto q3 = 60 * km / 2 * h;                               //
    Compile-time error

template<typename T>
auto make_length(T v) { return v * si::metre; }

auto v = 42 * m;
quantity<si::metre, int> q1 = make_length(v);           // Compile-time error
quantity<isq::length[si::metre]> q2 = make_length(v);   // Compile-time error
QuantityOf<isq::length> q3 = make_length(v);           // Compile-time error

template<typename T>
QuantityOf<isq::length> auto make_length(T v) { return v * si::metre; }

auto v = 42 * m;
quantity q = make_length(v); // Compile-time error

template<Representation T>
auto make_length(T v) { return v * si::metre; }

auto v = 42 * m;
quantity q = make_length(v); // Compile-time error

```

§ 9.4 Limitations of systems of quantities

As stated before, modeling systems of quantities and [Various quantities of the same kind](#) significantly improves the safety of the project. However, it is essential to mention here that such modeling is not ideal and there might be some pitfalls and surprises associated with some corner cases.

§ 9.4.1 Allowing irrational quantity combinations

Everyone probably agrees that multiplying two lengths is an area, and that area should be implicitly convertible to the result of such a multiplication:

```

static_assert(implicitly_convertible(isq::length * isq::length, isq::area));
static_assert(implicitly_convertible(isq::area, isq::length * isq::length));

```

Also, probably no one would be surprised by the fact that the multiplication of width and height is also convertible to area. Still, the reverse operation is not valid in this case. Not every area is an area over width and height, so we need an explicit cast to make such a conversion:

```

static_assert(implicitly_convertible(isq::width * isq::height, isq::area));
static_assert(!implicitly_convertible(isq::area, isq::width * isq::height));
static_assert(explicitly_convertible(isq::area, isq::width * isq::height));

```

However, it might be surprising to some that the similar behavior will also be observed for the product of two heights:

```

static_assert(implicitly_convertible(isq::height * isq::height, isq::area));
static_assert(!implicitly_convertible(isq::area, isq::height * isq::height));
static_assert(explicitly_convertible(isq::area, isq::height * isq::height));

```

For humans, it is hard to imagine how two heights form an area, but the library's logic has no way to prevent such operations.

§ 9.4.2 Quantities of dimension one

Some pitfalls might also arise when dealing with quantities of dimension one (also known as dimensionless quantities).

If we divide two quantities of the same kind, we end up with a quantity of dimension one. For example, we can divide two lengths to get a slope of the ramp or two durations to get the clock accuracy. Those ratios mean something fundamentally different, but from the dimensional analysis standpoint, they are mutually comparable.

The above means that the following code is valid:

```
quantity q1 = 1 * m / (10 * m) + 1 * us / (1 * h);
quantity q2 = isq::length(1 * m) / isq::length(10 * m) + isq::time(1 * us) / isq::time(1 * h);
quantity q3 = isq::height(1 * m) / isq::length(10 * m) + isq::time(1 * us) / isq::time(1 * h);
```

The fact that the above code compiles fine might again be surprising to some users of such a library. The result of all such quantity equations is a dimensionless quantity as it is the root of this hierarchy tree.

Now, let's try to convert such results to some quantities of dimension one. The `q1` was obtained from the expression that only used units in the equation, which means that the actual result of it is a quantity of `kind_of<dimensionless>`, which behaves like any quantity from the tree. Because of it, all of the below will compile for `q1`:

```
quantity<si::metre / si::metre> ok1 = q1;
quantity<(isq::length / isq::length)[m / m]> ok2 = q1;
quantity<(isq::height / isq::length)[m / m]> ok3 = q1;
```

For `q2` and `q3`, the two first conversions also succeed. The first one passes because all quantities of a kind are convertible to such kind. The type of the second quantity is `quantity<dimensionless[one]>` in disguise. Such a quantity is a root of the kind tree, so again, all the quantities from such a tree are convertible to it.

The third conversion fails in both cases, though. Not every dimensionless quantity is a result of dividing height and length, so an explicit conversion would be needed to force it to work.

```
quantity<si::metre / si::metre> ok4 = q2;
quantity<(isq::length / isq::length)[m / m]> ok5 = q2;
quantity<(isq::height / isq::length)[m / m]> bad1 = q2;

quantity<si::metre / si::metre> ok6 = q3;
quantity<(isq::length / isq::length)[m / m]> ok7 = q3;
quantity<(isq::height / isq::length)[m / m]> bad2 = q3;
```

§ 9.5 Temperatures

Temperature support is one the most challenging parts of any physical quantities and units library design. This is why it is probably reasonable to dedicate a chapter to this subject to describe how they are intended to work and what are the potential pitfalls or surprises.

First, let's run the following code:

```
quantity q1 = isq::thermodynamic_temperature(30. * K);
quantity q2 = isq::Celsius_temperature(30. * deg_C);

std::println("q1: {}, {}, {}", q1, q1.in(deg_C), q1.in(deg_F));
std::println("q2: {}, {}, {}", q2.in(K), q2, q2.in(deg_F));
```

This outputs:

```
q1: 30 K, 30 °C, 54 °F
q2: 30 K, 30 °C, 54 °F
```

Also doing the following:

```
quantity q3 = isq::Celsius_temperature(q1);
quantity q4 = isq::thermodynamic_temperature(q2);
```

outputs:

q3: 30 K

q4: 30 °C

Even though the [ISO 80000] provides dedicated quantity types for thermodynamic temperature and Celsius temperature, it explicitly states in the description of the first one:

Differences of thermodynamic temperatures or changes may be expressed either in kelvin, symbol K, or in degrees Celsius, symbol °C

In the description of the second quantity type, we can read:

The unit degree Celsius is a special name for the kelvin for use in stating values of Celsius temperature. The unit degree Celsius is by definition equal in magnitude to the kelvin. A difference or interval of temperature may be expressed in kelvin or in degrees Celsius.

As the quantity is a differential quantity type, it is okay to use any temperature unit for those, and the results should differ only by the conversion factor. No offset should be applied here to convert between the origins of different unit scales.

It is important to mention here that the existence of Celsius temperature quantity type in [ISO 80000] is controversial.

[ISO 80000] (part 1) says:

The system of quantities presented in this document is named the International System of Quantities (ISQ), in all languages. This name was not used in ISO 31 series, from which the present harmonized series has evolved. However, the ISQ does appear in ISO/IEC Guide 99 and is the system of quantities underlying the International System of Units, denoted “SI”, in all languages according to the SI Brochure.

According to the [ISO/IEC Guide 99], a system of quantities is a “set of quantities together with a set of non-contradictory equations relating those quantities”. It also defines the system of units as “set of base units and derived units, together with their multiples and submultiples, defined in accordance with given rules, for a given system of quantities”.

To say it explicitly, the system of quantities should not assume or use any specific units in its definitions. It is essential as various systems of units can be defined on top of it, and none of those should be favored.

However, the Celsius temperature quantity type is defined in [ISO 80000] (part 5) as:

temperature difference from the thermodynamic temperature of the ice point is called the Celsius temperature , which is defined by the quantity equation:

where T is thermodynamic temperature (item 5-1) and

Celsius temperature is an exceptional quantity in the ISQ as it uses specific SI units in its definition. This breaks the direction of dependencies between systems of quantities and units and imposes significant implementation issues.

As [mp-units] implementation clearly distinguishes between systems of quantities and units and assumes that the latter directly depends on the former, this quantity definition does not enforce any units or offsets. It is defined as just a more specialized quantity of the kind of thermodynamic temperature. We have added the Celsius temperature quantity type for completeness and to gain more experience with it. Still, maybe a good decision would be to skip it in the standardization process not to confuse users.

After quoting the official definitions and terms and presenting how quantities work, let's discuss quantity points. Those describe specific points and are measured relative to a provided origin:

```
quantity_point qp1 = si::absolute_zero + isq::thermodynamic_temperature(300. * K);
quantity_point qp2 = si::ice_point + isq::Celsius_temperature(30. * deg_C);
```

The above provides two different temperature points. The first one is measured as a relative quantity to the absolute zero (0 K), and the second one stores the value relative to the ice point being the beginning of the degree Celsius scale.

It is essential to understand that the origins of quantity points will not change if we convert their units:

```
quantity_point qp3 = qp1.in(deg_C);
quantity_point qp4 = qp2.in(K);

static_assert(std::is_same_v<decltype(qp3.point_origin), decltype(si::absolute_zero)>);
static_assert(std::is_same_v<decltype(qp4.point_origin), decltype(si::ice_point)>);
```

If we want to obtain the values of quantities in specific units relative to the origins of their scales, we have to be explicit:

```
std::println("qp1: {}, {}, {}",
             qp1.quantity_from(si::absolute_zero),
             qp1.quantity_from(si::ice_point).in(deg_C),
             qp1.quantity_from(usc::zero_Fahrenheit).in(deg_F));
std::println("qp2: {}, {}, {}",
             qp2.quantity_from(si::absolute_zero).in(K),
             qp2.quantity_from(si::ice_point),
             qp2.quantity_from(usc::zero_Fahrenheit).in(deg_F));
```

The above outputs:

```
qp1: 300 K, 26.85 °C, 80.33 °F
qp2: 303.15 K, 30 °C, 86 °F
```

Of course, all other combinations are also possible. For example, nothing should prevent us from checking how many degrees of Celsius are there starting from the zero_Fahrenheit for a quantity point using kelvins with the absolute_zero origin:

```
quantity q = qp1.quantity_from(usc::zero_Fahrenheit).in(deg_C);
```

Please also note that there is no text output support for quantity points in the [\[mp-units\]](#) library.

§ 9.6 Structural types

The quantity and quantity_point class templates are structural types to allow them to be passed as template arguments. For example, we can write the following:

```
constexpr struct amsterdam_sea_level : absolute_point_origin<isq::altitude> {
} amsterdam_sea_level;

constexpr struct mediterranean_sea_level : relative_point_origin<amsterdam_sea_level +
    isq::altitude(-27 * cm)> {
} mediterranean_sea_level;

using altitude_DE = quantity_point<isq::altitude[m], amsterdam_sea_level>;
using altitude_CH = quantity_point<isq::altitude[m], mediterranean_sea_level>;
```

Unfortunately, current language rules require that all member data of a structural type are public. This could be considered a safety issue. We try really hard to provide unit-safe interfaces, but at the same time expose the public “naked” data member that can be freely read or manipulated by anyone.

Hopefully, this requirement on structural types will be relaxed before the library gets standardized.

§ 10 Acknowledgements

Special thanks and recognition goes to [Epam Systems](#) for supporting Mateusz's membership in the ISO C++ Committee and the production of this proposal.

We would also like to thank Peter Sommerlad for providing valuable feedback that helped us shape the final version of this document.

§ 11 References

[Ariane flight V88] Ariane flight V88.

https://en.wikipedia.org/wiki/Ariane_flight_V88

[Boost.Units] Matthias C. Schabel and Steven Watanabe. Boost.Units.

https://www.boost.org/doc/libs/1_83_0/doc/html/boost_units.html

[Clarence] Steve Chawkins. Mismeasure for Measure.

<https://www.latimes.com/archives/la-xpm-2001-feb-09-me-23253-story.html>

[Columbus] Christopher Columbus.

https://en.wikipedia.org/wiki/Christopher_Columbus

[Disney] Cause of the Space Mountain Incident Determined at Tokyo Disneyland Park.

https://web.archive.org/web/20040209033827/http://www.olc.co.jp/news/20040121_01en.html

[Flight 6316] Korean Air Flight 6316 MD-11, Shanghai, China - April 15, 1999.

https://web.archive.org/web/20210917190721/https://www.nts.gov/news/press-releases/Pages/Korean_Air_Flight_6316_MD-11_Shanghai_China_-_April_15_1999.aspx

[Gimli Glider] Gimli Glider.

https://en.wikipedia.org/wiki/Gimli_Glider

[Hochrheinbrücke] An embarrassing discovery during the construction of a bridge.

https://www.normaalamsterdamspeil.nl/wp-content/uploads/2015/03/website_bridge.pdf

[ISO 80000] ISO80000: Quantities and units.

<https://www.iso.org/standard/76921.html>

[ISO/IEC Guide 99] ISO/IEC Guide 99: International vocabulary of metrology — Basic and general concepts and associated terms (VIM).

<https://www.iso.org/obp/ui#iso:std:iso-iec:guide:99>

[JCGM 200:2012] International vocabulary of metrology - Basic and general concepts and associated terms (VIM) (JCGM 200:2012, 3rd edition).

<https://jcg.m.bipm.org/vim/en>

[JSR 385] Units of Measurement.

<https://unitsofmeasurement.github.io/indriya>

[Mars Orbiter] Mars Climate Orbiter.

https://en.wikipedia.org/wiki/Mars_Climate_Orbiter

[Medication dose errors] Alma Mulac, Ellen Hagesaether, and Anne Gerd Granas. Medication dose calculation errors and other numeracy mishaps in hospitals: Analysis of the nature and enablers of incident reports.

<https://onlinelibrary.wiley.com/doi/10.1111/jan.15072>

[mp-units] mp-units - A Physical Quantities and Units library for C++.

<https://mpusz.github.io/mp-units>

[nholthaus/units] UNITS.

<https://github.com/nholthaus/units>

[P2980R0] Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Charles Hogg, Nicolas Holthaus, Roth Michaels, Vincent Reverdy. 2023-10-15. A motivation, scope, and plan for a physical quantities and units library.

<https://wg21.link/p2980r0>

[P2981R0] Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña. 2023-10-15. Improving our safety with a physical quantities and units library.

<https://wg21.link/p2981r0>

[Pint] Pint: makes units easy.

<https://pint.readthedocs.io/en/stable/index.html>

[Stonehenge] Tim Robey. Tiny stones, giant laughs: the story behind Spinal Tap's Stonehenge.

<https://www.telegraph.co.uk/films/2020/05/01/tiny-stones-giant-laughs-story-behind-spinal-taps-stonehenge>

[The Guardian] Charles Pensulo. Record Heat: Malawi swelters with temperatures nearly 68F above average.

<https://randomascii.wordpress.com/2023/10/17/localization-failure-temperature-is-hard>

[Value Category Is Not Lifetime] Arthur O'Dwyer. Value Category Is Not Lifetime.

<https://quuxplusone.github.io/blog/2019/03/11/value-category-is-not-lifetime/>

[Vasa] Rhitu Chatterjee and Lisa Mullins. New Clues Emerge in Centuries-Old Swedish Shipwreck.

<https://theworld.org/stories/2012-02-23/new-clues-emerge-centuries-old-swedish-shipwreck>

[Wild Rice] Manufacturers, exporters think metric.

<https://www.bizjournals.com/eastbay/stories/2001/07/09/focus3.html>